

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and [Amazon](http://Amazon.com).

Bayes's Theorem

In the previous chapter, we derived Bayes's Theorem:

$$P(A|B) = \frac{P(A)P(B|A)}{P(B)}$$

As an example, we used data from the General Social Survey and Bayes's Theorem to compute conditional probabilities. But since we had the complete dataset, we didn't really need Bayes's Theorem. It was easy enough to compute the left side of the equation directly, and no easier to compute the right side.

But often we don't have a complete dataset, and in that case Bayes's Theorem is more useful. In this chapter, we'll use it to solve several more challenging problems related to conditional probability.

The Cookie Problem

We'll start with a thinly disguised version of an urn problem:

Suppose there are two bowls of cookies.

- *Bowl 1 contains 30 vanilla cookies and 10 chocolate cookies.*
- *Bowl 2 contains 20 vanilla cookies and 20 chocolate cookies.*

Now suppose you choose one of the bowls at random and, without looking, choose a cookie at random. If the cookie is vanilla, what is the probability that it came from Bowl 1?

What we want is the conditional probability that we chose from Bowl 1 given that we got a vanilla cookie, $P(B_1|V)$.

But what we get from the statement of the problem is:

- The conditional probability of getting a vanilla cookie, given that we chose from Bowl 1, $P(V|B_1)$ and
- The conditional probability of getting a vanilla cookie, given that we chose from Bowl 2, $P(V|B_2)$.

Bayes's Theorem tells us how they are related:

$$P(B_1|V) = \frac{P(B_1) P(V|B_1)}{P(V)}$$

The term on the left is what we want. The terms on the right are:

- $P(B_1)$, the probability that we chose Bowl 1, unconditioned by what kind of cookie we got. Since the problem says we chose a bowl at random, we assume $P(B_1) = 1/2$.
- $P(V|B_1)$, the probability of getting a vanilla cookie from Bowl 1, which is $3/4$.
- $P(V)$, the probability of drawing a vanilla cookie from either bowl.

To compute $P(V)$, we can use the law of total probability:

$$P(V) = P(B_1) P(V|B_1) + P(B_2) P(V|B_2)$$

Plugging in the numbers from the statement of the problem, we have

$$P(V) = (1/2) (3/4) + (1/2) (1/2) = 5/8$$

We can also compute this result directly, like this:

- Since we had an equal chance of choosing either bowl and the bowls contain the same number of cookies, we had the same chance of choosing any cookie.
- Between the two bowls there are 50 vanilla and 30 chocolate cookies, so $P(V) = 5/8$.

Finally, we can apply Bayes's Theorem to compute the posterior probability of Bowl 1:

$$P(B_1|V) = (1/2) (3/4) / (5/8) = 3/5$$

This example demonstrates one use of Bayes's theorem: it provides a way to get from $P(B|A)$ to $P(A|B)$. This strategy is useful in cases like this where it is easier to compute the terms on the right side than the term on the left.

Diachronic Bayes

There is another way to think of Bayes's theorem: it gives us a way to update the probability of a hypothesis, H , given some body of data, D .

This interpretation is "diachronic", which means "related to change over time"; in this case, the probability of the hypotheses changes as we see new data.

Rewriting Bayes's theorem with H and D yields:

$$P(H|D) = \frac{P(H) P(D|H)}{P(D)}$$

In this interpretation, each term has a name:

- $P(H)$ is the probability of the hypothesis before we see the data, called the prior probability, or just **prior**.
- $P(H|D)$ is the probability of the hypothesis after we see the data, called the **posterior**.
- $P(D|H)$ is the probability of the data under the hypothesis, called the **likelihood**.
- $P(D)$ is the **total probability of the data**, under any hypothesis.

Sometimes we can compute the prior based on background information. For example, the cookie problem specifies that we choose a bowl at random with equal probability.

In other cases the prior is subjective; that is, reasonable people might disagree, either because they use different background information or because they interpret the same information differently.

The likelihood is usually the easiest part to compute. In the cookie problem, we are given the number of cookies in each bowl, so we can compute the probability of the data under each hypothesis.

Computing the total probability of the data can be tricky. It is supposed to be the probability of seeing the data under any hypothesis at all, but it can be hard to nail down what that means.

Most often we simplify things by specifying a set of hypotheses that are:

- Mutually exclusive, which means that only one of them can be true, and
- Collectively exhaustive, which means one of them must be true.

When these conditions apply, we can compute $P(D)$ using the law of total probability. For example, with two hypotheses, H_1 and H_2 :

$$P(D) = P(H_1) P(D|H_1) + P(H_2) P(D|H_2)$$

And more generally, with any number of hypotheses:

$$P(D) = \sum_i P(H_i) P(D|H_i)$$

The process in this section, using data and a prior probability to compute a posterior probability, is called a **Bayesian update**.

Bayes Tables

A convenient tool for doing a Bayesian update is a Bayes table. You can write a Bayes table on paper or use a spreadsheet, but in this section I'll use a Pandas `DataFrame`.

First I'll make empty `DataFrame` with one row for each hypothesis:

```
import pandas as pd

table = pd.DataFrame(index=['Bowl 1', 'Bowl 2'])
```

Now I'll add a column to represent the priors:

```
table['prior'] = 1/2, 1/2
table
```

	prior
Bowl 1	0.5
Bowl 2	0.5

And a column for the likelihoods:

```
table['likelihood'] = 3/4, 1/2
table
```

	prior	likelihood
Bowl 1	0.5	0.75
Bowl 2	0.5	0.50

Here we see a difference from the previous method: we compute likelihoods for both hypotheses, not just Bowl 1:

- The chance of getting a vanilla cookie from Bowl 1 is 3/4.
- The chance of getting a vanilla cookie from Bowl 2 is 1/2.

You might notice that the likelihoods don't add up to 1. That's OK; each of them is a probability conditioned on a different hypothesis. There's no reason they should add up to 1 and no problem if they don't.

The next step is similar to what we did with Bayes's Theorem; we multiply the priors by the likelihoods:

```
table['unnorm'] = table['prior'] * table['likelihood']
table
```

	prior	likelihood	unnorm
Bowl 1	0.5	0.75	0.375
Bowl 2	0.5	0.50	0.250

I call the result `unnorm` because these values are the "unnormalized posteriors". Each of them is the product of a prior and a likelihood:

$$P(H_i) P(D|H_i)$$

which is the numerator of Bayes's Theorem. If we add them up, we have

$$P(H_1) P(D|H_1) + P(H_2) P(D|H_2)$$

which is the denominator of Bayes's Theorem, $P(D)$.

So we can compute the total probability of the data like this:

```
prob_data = table['unnorm'].sum()
prob_data
```

```
np.float64(0.625)
```

Notice that we get $5/8$, which is what we got by computing $P(D)$ directly.

And we can compute the posterior probabilities like this:

```
table['posterior'] = table['unnorm'] / prob_data
table
```

	prior	likelihood	unnorm	posterior
Bowl 1	0.5	0.75	0.375	0.6
Bowl 2	0.5	0.50	0.250	0.4

The posterior probability for Bowl 1 is 0.6, which is what we got using Bayes's Theorem explicitly. As a bonus, we also get the posterior probability of Bowl 2, which is 0.4.

When we add up the unnormalized posteriors and divide through, we force the posteriors to add up to 1. This process is called "normalization", which is why the total probability of the data is also called the "normalizing constant".

The Dice Problem

A Bayes table can also solve problems with more than two hypotheses. For example:

Suppose I have a box with a 6-sided die, an 8-sided die, and a 12-sided die. I choose one of the dice at random, roll it, and report that the outcome is a 1. What is the probability that I chose the 6-sided die?

In this example, there are three hypotheses with equal prior probabilities. The data is my report that the outcome is a 1.

If I chose the 6-sided die, the probability of the data is $1/6$. If I chose the 8-sided die, the probability is $1/8$, and if I chose the 12-sided die, it's $1/12$.

Here's a Bayes table that uses integers to represent the hypotheses:

```
table2 = pd.DataFrame(index=[6, 8, 12])
```

I'll use fractions to represent the prior probabilities and the likelihoods. That way they don't get rounded off to floating-point numbers.

```

from fractions import Fraction

table2['prior'] = Fraction(1, 3)
table2['likelihood'] = Fraction(1, 6), Fraction(1, 8), Fraction(1, 12)
table2

```

	prior	likelihood
6	$1/3$	$1/6$
8	$1/3$	$1/8$
12	$1/3$	$1/12$

Once you have priors and likelihoods, the remaining steps are always the same, so I'll put them in a function:

```

def update(table):
    """Compute the posterior probabilities."""
    table['unnorm'] = table['prior'] * table['likelihood']
    prob_data = table['unnorm'].sum()
    table['posterior'] = table['unnorm'] / prob_data
    return prob_data

```

And call it like this.

```

prob_data = update(table2)

```

Here is the final Bayes table:

```

table2

```

	prior	likelihood	unnorm	posterior
6	$1/3$	$1/6$	$1/18$	$4/9$
8	$1/3$	$1/8$	$1/24$	$1/3$
12	$1/3$	$1/12$	$1/36$	$2/9$

The posterior probability of the 6-sided die is $4/9$, which is a little more than the probabilities for the other dice, $3/9$ and $2/9$. Intuitively, the 6-sided die is the most likely because it had the highest likelihood of producing the outcome we saw.

The Monty Hall Problem

Next we'll use a Bayes table to solve one of the most contentious problems in probability.

The Monty Hall problem is based on a game show called *Let's Make a Deal*. If you are a contestant on the show, here's how the game works:

- The host, Monty Hall, shows you three closed doors -- numbered 1, 2, and 3 -- and tells you that there is a prize behind each door.
- One prize is valuable (traditionally a car), the other two are less valuable (traditionally goats).
- The object of the game is to guess which door has the car. If you guess right, you get to keep the car.

Suppose you pick Door 1. Before opening the door you chose, Monty opens Door 3 and reveals a goat. Then Monty offers you the option to stick with your original choice or switch to the remaining unopened door.

To maximize your chance of winning the car, should you stick with Door 1 or switch to Door 2?

To answer this question, we have to make some assumptions about the behavior of the host:

1. Monty always opens a door and offers you the option to switch.
2. He never opens the door you picked or the door with the car.
3. If you choose the door with the car, he chooses one of the other doors at random.

Under these assumptions, you are better off switching. If you stick, you win $1/3$ of the time. If you switch, you win $2/3$ of the time.

If you have not encountered this problem before, you might find that answer surprising. You would not be alone; many people have the strong intuition that it doesn't matter if you stick or switch. There are two doors left, they reason, so the chance that the car is behind Door A is 50%. But that is wrong.

To see why, it can help to use a Bayes table. We start with three hypotheses: the car might be behind Door 1, 2, or 3. According to the statement of the problem, the prior probability for each door is $1/3$.

```
table3 = pd.DataFrame(index=['Door 1', 'Door 2', 'Door 3'])
table3['prior'] = Fraction(1, 3)
table3
```


	prior
Door 1	1/3
Door 2	1/3
Door 3	1/3

The data is that Monty opened Door 3 and revealed a goat. So let's consider the probability of the data under each hypothesis:

- If the car is behind Door 1, Monty chooses Door 2 or 3 at random, so the probability he opens Door 3 is $1/2$.
- If the car is behind Door 2, Monty has to open Door 3, so the probability of the data under this hypothesis is 1.
- If the car is behind Door 3, Monty does not open it, so the probability of the data under this hypothesis is 0.

Here are the likelihoods.

```
table3['likelihood'] = Fraction(1, 2), 1, 0
table3
```

	prior	likelihood
Door 1	1/3	1/2
Door 2	1/3	1
Door 3	1/3	0

Now that we have priors and likelihoods, we can use `update` to compute the posterior probabilities.

```
update(table3)
table3
```

	prior	likelihood	unnorm	posterior
Door 1	1/3	1/2	1/6	1/3
Door 2	1/3	1	1/3	2/3
Door 3	1/3	0	0	0

After Monty opens Door 3, the posterior probability of Door 1 is $1/3$; the posterior probability of Door 2 is $2/3$. So you are better off switching from Door 1 to Door 2.

As this example shows, our intuition for probability is not always reliable. Bayes's Theorem can help by providing a divide-and-conquer strategy:

1. First, write down the hypotheses and the data.
2. Next, figure out the prior probabilities.
3. Finally, compute the likelihood of the data under each hypothesis.

The Bayes table does the rest.

Summary

In this chapter we solved the Cookie Problem using Bayes's theorem explicitly and using a Bayes table. There's no real difference between these methods, but the Bayes table can make it easier to compute the total probability of the data, especially for problems with more than two hypotheses.

Then we solved the Dice Problem, which we will see again in the next chapter, and the Monty Hall problem, which you might hope you never see again.

If the Monty Hall problem makes your head hurt, you are not alone. But I think it demonstrates the power of Bayes's Theorem as a divide-and-conquer strategy for solving tricky problems. And I hope it provides some insight into *why* the answer is what it is.

When Monty opens a door, he provides information we can use to update our belief about the location of the car. Part of the information is obvious. If he opens Door 3, we know the car is not behind Door 3. But part of the information is more subtle. Opening Door 3 is more likely if the car is behind Door 2, and less likely if it is behind Door 1. So the data is evidence in favor of Door 2. We will come back to this notion of evidence in future chapters.

In the next chapter we'll extend the Cookie Problem and the Dice Problem, and take the next step from basic probability to Bayesian statistics.

But first, you might want to work on the exercises.

Exercises

Exercise: Suppose you have two coins in a box. One is a normal coin with heads on one side and tails on the other, and one is a trick coin with heads on both sides. You choose a coin at random and see that one of the sides is heads. What is the probability that you chose the trick coin?

Solution

```
table4 = pd.DataFrame(index=['Normal', 'Trick'])
table4['prior'] = 1/2
table4['likelihood'] = 1/2, 1

update(table4)
table4
```

	prior	likelihood	unnorm	posterior
Normal	0.5	0.5	0.25	0.333333
Trick	0.5	1.0	0.50	0.666667

Exercise: Suppose you meet someone and learn that they have two children. You ask if either child is a girl and they say yes. What is the probability that both children are girls?

Hint: Start with four equally likely hypotheses.

Solution

```
table5 = pd.DataFrame(index=['GG', 'GB', 'BG', 'BB'])
table5['prior'] = 1/4
table5['likelihood'] = 1, 1, 1, 0

update(table5)
table5
```

	prior	likelihood	unnorm	posterior
GG	0.25	1	0.25	0.333333
GB	0.25	1	0.25	0.333333
BG	0.25	1	0.25	0.333333
BB	0.25	0	0.00	0.000000

Exercise: There are many variations of the Monty Hall problem.

For example, suppose Monty always chooses Door 2 if he can, and only chooses Door 3 if he has to (because the car is behind Door 2).

If you choose Door 1 and Monty opens Door 2, what is the probability the car is behind Door 3?

If you choose Door 1 and Monty opens Door 3, what is the probability the car is behind Door 2?

Solution

If the car is behind Door 1, Monty would always open Door 2
If the car was behind Door 2, Monty would have opened Door 3
If the car is behind Door 3, Monty would always open Door 2

```
table6 = pd.DataFrame(index=['Door 1', 'Door 2', 'Door 3'])
```

```
table6['prior'] = 1/3
```

```
table6['likelihood'] = 1, 0, 1
```

```
update(table6)
```

```
table6
```

	prior	likelihood	unnorm	posterior
Door 1	0.333333	1	0.333333	0.5
Door 2	0.333333	0	0.000000	0.0
Door 3	0.333333	1	0.333333	0.5

Solution

If the car is behind Door 1, Monty would have opened Door 2
If the car is behind Door 2, Monty would always open Door 3
If the car is behind Door 3, Monty would have opened Door 2

```
table7 = pd.DataFrame(index=['Door 1', 'Door 2', 'Door 3'])
```

```
table7['prior'] = 1/3
```

```
table7['likelihood'] = 0, 1, 0
```

```
update(table7)
```

```
table7
```

	prior	likelihood	unnorm	posterior
Door 1	0.333333	0	0.000000	0.0
Door 2	0.333333	1	0.333333	1.0
Door 3	0.333333	0	0.000000	0.0

Exercise: M&M's are small candy-coated chocolates that come in a variety of colors.

Mars, Inc., which makes M&M's, changes the mixture of colors from time to time. In 1995, they introduced blue M&M's.

- In 1994, the color mix in a bag of plain M&M's was 30% Brown, 20% Yellow, 20% Red, 10% Green, 10% Orange, 10% Tan.
- In 1996, it was 24% Blue, 20% Green, 16% Orange, 14% Yellow, 13% Red, 13% Brown.

Suppose a friend of mine has two bags of M&M's, and he tells me that one is from 1994 and one from 1996. He won't tell me which is which, but he gives me one M&M from each bag. One is yellow and one is green. What is the probability that the yellow one came from the 1994 bag?

Hint: The trick to this question is to define the hypotheses and the data carefully.

Solution

Hypotheses:

A: yellow from 94, green from 96

B: yellow from 96, green from 94

```
table8 = pd.DataFrame(index=['A', 'B'])
```

```
table8['prior'] = 1/2
```

```
table8['likelihood'] = 0.2*0.2, 0.14*0.1
```

```
update(table8)
```

```
table8
```

	prior	likelihood	unnorm	posterior
A	0.5	0.040	0.020	0.740741
B	0.5	0.014	0.007	0.259259

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)